Attention Is All You Need

Transformers can be thought of as a new `op` which replaces recurrence and convolution with attention mechanisms. A scaled dot-product attention unit, extended via multi-head attention captures diverse relationships in parallel. Positional encodings encode sequence order information. Stacked encoder and decoder layers interleave `(1)` attention, `(2)` feed-forward sublayers, `(3)` residual connections, and `(4)` layer normalization to build compositional representations. Masking - in the decoder - prevents future information leakage (though bidirectional models do not preclude bidirectional dependencies at train time).

Scaled Dot-Product Attention

Scaled dot-product attention computes how each element in a sequence should attend to every other element. It transforms the input embeddings into three distinct spaces — queries (Q), keys (K), and values (V). These spaces are then scored and weighted to produce context-aware representations.

Given query $\mathbf{Q}$, key $\mathbf{K}$, and value $\mathbf{V}$ matrices, attention is computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\big(\tfrac{QK^\top}{\sqrt{d_k}}\big)\, V.$$

- **Query–Key Matching**: The query asks the question: *What information do I need?*. Keys are potential answers. Attention scores tell us which keys best match our query. The matrix multiplication $QK^\top$ produces raw similarity scores between queries and keys.
- **Weighted Context**: The resulting softmax weights act like a "soft alignment" over positions, producing a context vector that blends information from all tokens according to relevance.
- Dividing by $\sqrt{d_k}$ stabilizes gradients. Without scaling, dot products grow large in high dimensions, pushing softmax into regions with extremely small gradients. The $\sqrt{d_k}$ normalization keeps the logits in a more moderate range, allowing each token to gather information from all others.

Multi-Head Attention

Rather than a single attention head, the Transformer uses h parallel heads, each with its own learned linear projections for Q, K, and V. Heads run simultaneously to capture different types of relationships (e.g., syntax vs. semantics), then concatenate their outputs and linearly project back to the model space. This enriches representation power without significant computational overheads.

1. Project Q, K, V into h subspaces via linear layers.
2. Compute scaled dot-product attention independently for each head.

3. Concatenate the h head outputs and linearly project back to the model dimension.

This allows the model to attend jointly to information from different representation subspaces at different positions.

Positional Encoding

Transformers apply self-attention in parallel across tokens, making them inherently permutation-invariant. Without encoding any position information, the sequence `a,b,c,d` and the sequence `d,c,a,b` would produce identical attention patterns. But `seq2seq` models need to be aware of order. Positional encodings remedy this by giving each token a unique position signal added to its embedding before attention layers.

The original paper adds sinusoidal functions of differing frequencies that are summed with token embeddings. Alternative approaches include learned positional embeddings (where positions are end2end trained vectors) used in BERT and GPT models, and Rotary Positional Embeddings (where position is encoded via rotations in embedding space) used in LLaMA and PALM.

Encoder & Decoder Stacks

- **Encoder**: Composed of N identical layers, each with two sub-layers:
    1. Multi-head self-attention over the input tokens.
    2. Position-wise fully connected feed-forward network.
- **Decoder**: Also N layers, but each layer has three sub-layers:
    1. Masked multi-head self-attention (masking preserves the autoregressive property of not looking at future tokens).
    2. Multi-head attention over encoder outputs (allowing the decoder to "look" at the source).
    3. Feed-forward network.

In both encoders and decoders, residual connections around each sub-layer and layer normalization ensure stable training.

Feed-Forward Networks

Each position-wise feed-forward network takes the form:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)\, W_2 + b_2.$$

Since FFNs consist of fully connected layers, they make up the majority of model parameters.

Miscellaneous

- **Time Complexity**: Time complexity per layer per token is $O(n^2 d)$ due to $QK^\top$ dot products (with sequence length n and model dimension d).
- **Memory complexity**: Model complexity is $O(n^2)$, limiting very long sequences without modifications (cf. sparse attention).
- In the Transformer's **encoder** and **decoder** layers:
  - **Self-Attention**: Queries, keys, and values all come from the same sequence (enable each position to attend to every other).
  - **Encoder–Decoder Attention**: In the decoder, queries come from the previous decoder layer, while keys and values come from the encoder output (allowing the decoder to look back at the source).
  - **Masking**: In the decoder's self-attention, a causal mask prevents attending to future tokens, preserving autoregressive decoding.

The Illustrated Transformer

Transformers comprise of stacked encoder and decoder blocks, each built from (1) multi-headed self-attention to capture token-wise dependencies, (2) position-wise feed-forward networks to introduce nonlinearity, (3) residual connections and layer normalization for stable deep stacking, and (4) positional encodings to inject order information. During decoding, an additional encoder–decoder attention layer lets the model align output tokens to encoded inputs.

Scaler Dot Product Self-Attention

Each token's embedding $x_i \in \mathbb{R}^{d_{\text{model}}}$ is projected into queries, keys, and values: $Q = XW^Q, \quad K = XW^K, \quad V = XW^V$, with $W^Q, W^K, W^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$.

Attention scores are computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)\, V,$$

where scaling by $\sqrt{d_k}$ stabilizes gradients in high dimensions.

`Intuition`: $QK^\top$ tells us *where* to look, and V tells us *what* we see when we look there. After computing the attention scores using $QK^\top$, these scores are used to form a weighted sum of the V vectors. Each output vector is a blend of all input V vectors, where the contribution from each token is determined by its relevance to the query. While Q and K are used for calculating attention weights, V ensures that the semantic meaning from the input tokens is preserved and passed forward, filtered by the attention mechanism.

Multi-Head Attention

Rather than one, h parallel heads learn distinct projections $\{W_j^Q, W_j^K, W_j^V\}_{j=1}^h$, each producing an output Z_j. These are concatenated and linearly projected back to dimension d_{model}:

$$\text{MultiHead}(X) = \big[Z_1; \ldots; Z_h\big]\, W^O, \quad W^O \in \mathbb{R}^{h\cdot\, d_k \times d_{\text{model}}}.$$

This enables the model to capture multiple types of relationships (e.g. syntactic vs. semantic) in parallel.

Position-Wise Feed-Forward Network

After attention, each position independently passes through

$$\text{FFN}(x) = \max\big(0,\, xW_1 + b_1\big)\, W_2 + b_2,$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$.

Residuals & Normalization

Each sub-layer (attention or FFN) is wrapped with a **residual connection** and **layer normalization**: $\text{LayerNorm}\big(x + \text{Sublayer}(x)\big)$.

This preserves gradient flow in deep stacks (e.g. 6–12 layers) and prevents representations from drifting too far at each step.

Injecting Order: Positional Encoding

Self-attention is blind to token order. To remedy this, the model adds a deterministic positional-encoding matrix $\text{PE} \in \mathbb{R}^{n \times d_{\text{model}}}$ to the input embeddings $E \in \mathbb{R}^{n \times d_{\text{model}}}$. For position pos and dimension i: $\text{PE}(pos, 2i) = \sin\big(\frac{pos}{10000^{2i/d_{\text{model}}}}\big)$ and $\text{PE}(pos, 2i+1) = \cos\big(\frac{pos}{10000^{2i/d_{\text{model}}}}\big)$.

This creates unique, continuous patterns that let the model learn both **absolute** and **relative** positions—and generalize to sequences longer than those seen in training.

`NOTE`: Positional encodings are added (not concatenated) to token embeddings to inject word order information without increasing input dimensionality, keeping the model efficient and compatible with downstream layers. This addition intentionally "interferes" with the token embedding, but in a structured way that helps the model learn both what a token is and where it appears. Empirically, this blending of position and content works well, and the model can learn to untangle or emphasize or ignore position as needed. Later, learnt positional embeddings became common due to models such as BERT and GPT.

The Decoder's Cross-Attention

In each decoder layer, after masked self-attention (preventing "peeking" ahead), a second attention sublayer lets decoder queries attend to encoder outputs (keys/values). Formally, if Q_d comes from the decoder and K_e, V_e come from the encoder, then $\text{Attention}(Q_d, K_e, V_e) = \text{softmax}\big(Q_d K_e^\top / \sqrt{d_k}\big)\, V_e$.

This aligns generated tokens with relevant source tokens during translation.

Final Linear & Softmax

The decoder's top-layer outputs pass through a linear layer and softmax to produce token probabilities over the vocabulary, enabling autoregressive generation one token at a time.

References

1. Attention Is All You Need. arxiv, google-blog.
2. Transformer: A Novel Neural Network Architecture for Language Understanding. google-blog.
3. The Illustrated Transformer blog.